

Student 3 **DANIMIZA V** 192421363RC

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

arr = list(map(int, input().split()))

sorted_arr = selection_sort(arr)

print(sorted_arr)
```

INPUT

23 78 45 8 32 56

OUTPUT

[8, 23, 32, 45, 56, 78]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

arr = list(map(int, input().split()))

sorted_arr = insertion_sort(arr)

print(sorted_arr)
```

INPUT

8 5 7 1 9 3

OUTPUT

[1, 3, 5, 7, 8, 9]

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report
Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

    return arr

arr = list(map(int, input().split()))

sorted_arr = merge_sort(arr)

print(sorted_arr)
```

INPUT 38 27 43 3 9 82 10**OUTPUT** [3, 9, 10, 27, 38, 43, 82]

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
def is_diagonal_matrix(matrix):
    rows = len(matrix)
    cols = len(matrix[0])

    if rows != cols:
        return False

    for i in range(rows):
        for j in range(cols):
            if i != j and matrix[i][j] != 0:
                return False
    return True

input_str = input().strip()

rows = input_str[2:-2].split(",")
matrix = [[int(val) for val in row.split(",")] for row in rows]

if is_diagonal_matrix(matrix):
    print("Diagonal Matrix")
else:
    print("Not Diagonal Matrix")
```

INPUT [[1,0,0],[0,5,0],[0,0,8]]**OUTPUT** Diagonal Matrix**Q5. Row and Column Sums**

Score: 6.67 TC: 2/3 passed

CODE

```
input_str = input().strip()

rows_str = input_str[2:-2].split(",")
matrix = [[int(val) for val in row.split(",")] for row in rows_str]

rows = len(matrix)
cols = len(matrix[0])

row_sums = [sum(row) for row in matrix]

col_sums = [sum(matrix[r][c] for r in range(rows)) for c in range(cols)]

print("Row=", row_sums)
print("Col=", col_sums)
```

INPUT [[1,2],[3,4]]**OUTPUT** Row= [3, 7]
Col= [4, 6]

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
def max_subarray_sum(arr):
    max_so_far = arr[0]
    curr_max = arr[0]

    for i in range(1, len(arr)):
        curr_max = max(arr[i], curr_max + arr[i])
        max_so_far = max(max_so_far, curr_max)

    return max_so_far

arr = list(map(int, input().split()))

print(max_subarray_sum(arr))
```

INPUT

-2 1 -3 4 -1 2 1 -5 4

OUTPUT

6

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
def find_common_elements(input_str):

    parts = input_str.strip().split(';')

    set1 = set(map(int, parts[0].split()))
    set2 = set(map(int, parts[1].split()))
    set3 = set(map(int, parts[2].split()))

    common = sorted(list(set1 & set2 & set3))

    return common

input_str = input()

print(find_common_elements(input_str))
```

INPUT

1 2 3 ; 2 3 4 ; 2 5

OUTPUT

[2]

Q8. Matrix Multiplication

Score: 6.67 TC: 2/3 passed

CODE

```
def parse_matrix(s):
    s = s.strip()
    rows = s[2:-2].split(",")
    return [[int(x) for x in row.split(",")] for row in rows]

input_str = input().strip()

parts = input_str.split(" B=")
str_A = parts[0].replace("A=", "")
str_B = parts[1]

A = parse_matrix(str_A)
B = parse_matrix(str_B)

rows_A, cols_A = len(A), len(A[0])
rows_B, cols_B = len(B), len(B[0])

result = [[0] * cols_B for _ in range(rows_A)]
for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]

if len(result) == 1:
    print(result)
else:
    for i, row in enumerate(result):
        if i == 0:
            print(f"[{row}]")
        elif i == len(result) - 1:
            print(f"[{row}]")
        else:
            print(f"{row}")
```

INPUT

A=[[1,2],[3,4]] B=[[5,6],[7,8]]

OUTPUT

[[19, 22]
[43, 50]]

Q9. Sort Rotate Min Max

Score: 0.00 TC: 0/3 passed

CODE

```
# Read full input string
raw_input = input().strip()

# Extract list and k
if "k=" in raw_input:
    parts = raw_input.split("k=")
    k = int(parts[1].strip())
    list_str = parts[0].replace("List=", "").replace(",", "").strip()
else:
    list_str = raw_input
    k = 0

# Convert input to integer list
arr = [int(x) for x in list_str.split()]

# Sort list
sorted_arr = sorted(arr)

# Rotate list (Rotate left/shift so element at index k becomes first)
n = len(sorted_arr)
if n > 0:
    k = k % n
    rotated_arr = sorted_arr[k:] + sorted_arr[:k]
else:
    rotated_arr = []

# Find Min and Max
min_val = min(sorted_arr)
max_val = max(sorted_arr)

# Print without space after =
print("Sorted=", sorted_arr, sep="")
print("Rotated=", rotated_arr, sep="")
print("Min=", min_val, sep=" ")
print("Max=", max_val, sep=" ")
```

INPUT

List=5 2 8 1 4,k=2

OUTPUT

```
Sorted=[1, 2, 4, 5, 8]
Rotated=[4, 5, 8, 1, 2]
Min= 1
Max= 8
```

Q10. Sort Matrix Search

Score: 3.33 TC: 1/3 passed

CODE

```
raw_input = input().strip()

parts = raw_input.split(", Find=")
matrix_str = parts[0].replace("Matrix=", "").strip()
target = int(parts[1].strip())

rows_str = matrix_str[2:-2].split(",")
matrix = [[int(val) for val in row.split(",")] for row in rows_str]

rows = len(matrix)
cols = len(matrix[0])

flat_list = sorted([elem for row in matrix for elem in row])

sorted_matrix = []
for i in range(rows):
    sorted_matrix.append(flat_list[i * cols : (i + 1) * cols])

pos = None
for r in range(rows):
    for c in range(cols):
        if sorted_matrix[r][c] == target:
            pos = (r, c)
            break
    if pos:
        break

if pos:

    formatted_matrix = "[" + ", ".join(str(r) for r in sorted_matrix) + "]"
    print("Sorted=", formatted_matrix, sep=" ")
    print("Position=", pos, sep=" ")
else:
    print("Not Found")
```

INPUT

Matrix=[[9,3],[7,1]], Find=7

OUTPUTSorted= [[1, 3],[7, 9]]
Position= (1, 0)